

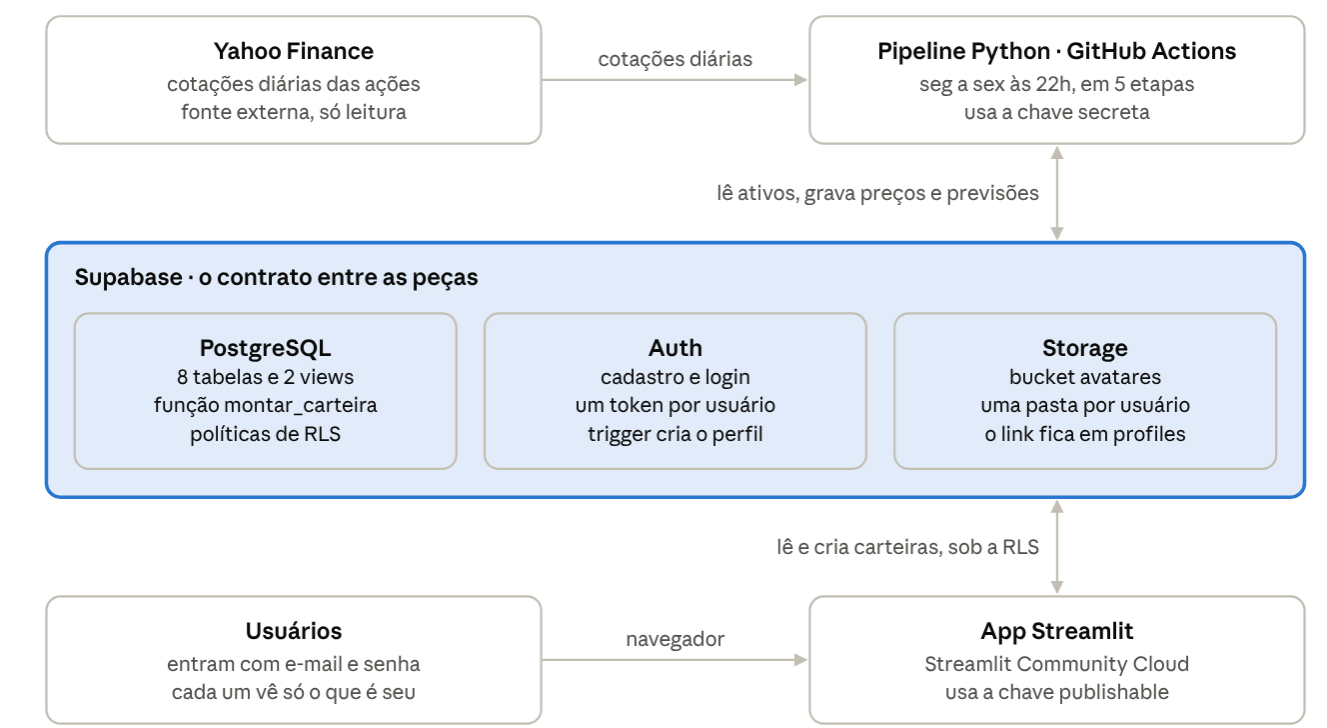
Monitor B3 — A arquitetura do app, explicada

Oct 1, 2026 · @Vahid Nikoofard

Visão geral

O Monitor B3 tem quatro peças, e as duas que têm código nunca conversam entre si: o pipeline Python **escreve** no banco, o app Streamlit **lê** do banco, e o Supabase fica no meio garantindo as regras.

O pipeline escreve, o app lê: o Supabase é o único ponto de contato



arquitetura · 4 peças, o Supabase no centro

Nenhuma seta liga o pipeline ao app: tudo o que um produz e o outro consome passa pelo Supabase.

Peça	Onde roda	Papel
Yahoo Finance	Internet	Fonte das cotações diárias
Pipeline Python	GitHub Actions, de segunda a sexta às 22h	Baixa preços, calcula indicadores, treina o XGBoost e grava as previsões

Peça	Onde roda	Papel
Supabase	Nuvem do Supabase (região São Paulo)	Banco PostgreSQL, login dos usuários e armazenamento das fotos
App Streamlit	Streamlit Community Cloud	Telas de ranking, carteiras, modelo e perfil

Essa separação é a ideia central do projeto. Como o banco é o único ponto de contato, cada peça pode ser trocada sem mexer nas outras: o frontend já existiu em Lovable e hoje é Streamlit, e o pipeline nem percebeu.

Um dia na vida do sistema

O sistema tem um relógio: uma vez por dia útil, à noite, o pipeline renova os dados; durante o dia, os usuários consomem o que foi gravado.

Quando	Quem age	O que acontece
22h, de segunda a sexta	GitHub Actions	Liga uma máquina Linux nova, roda o teste sem internet e depois as 5 etapas do pipeline, em 2 a 4 minutos
Logo depois	Supabase	Passa a ter os preços do dia, uma nova versão do modelo e o ranking do pregão
No dia seguinte	Usuário, pelo app	Entra, consulta o ranking e monta uma carteira, que fica "em andamento"
5 pregões depois	Pipeline	Calcula o retorno que de fato aconteceu e preenche <code>retorno_realizado</code> ; a carteira passa a mostrar o resultado
A qualquer momento	Professor, no Table Editor	Insere ou desativa uma ação em <code>ativos</code> ; vale a partir da próxima execução
Após 12 h sem acesso	Streamlit Community Cloud	O app hiberna; o próximo visitante espera cerca de um minuto para acordá-lo

Repare que ninguém precisa avisar ninguém: o app não sabe quando o pipeline rodou, apenas lê o que estiver no banco. Esse desacoplamento no tempo é o mesmo desacoplamento das peças, visto de outro ângulo.

O banco como contrato

O schema do Supabase é o contrato entre o pipeline e o app: os nomes de tabelas, colunas, views e funções são a única coisa que os dois lados precisam combinar. A construção passo a passo, com o diagrama das tabelas, está em

Monitor B3 — Construindo o banco no Supabase .

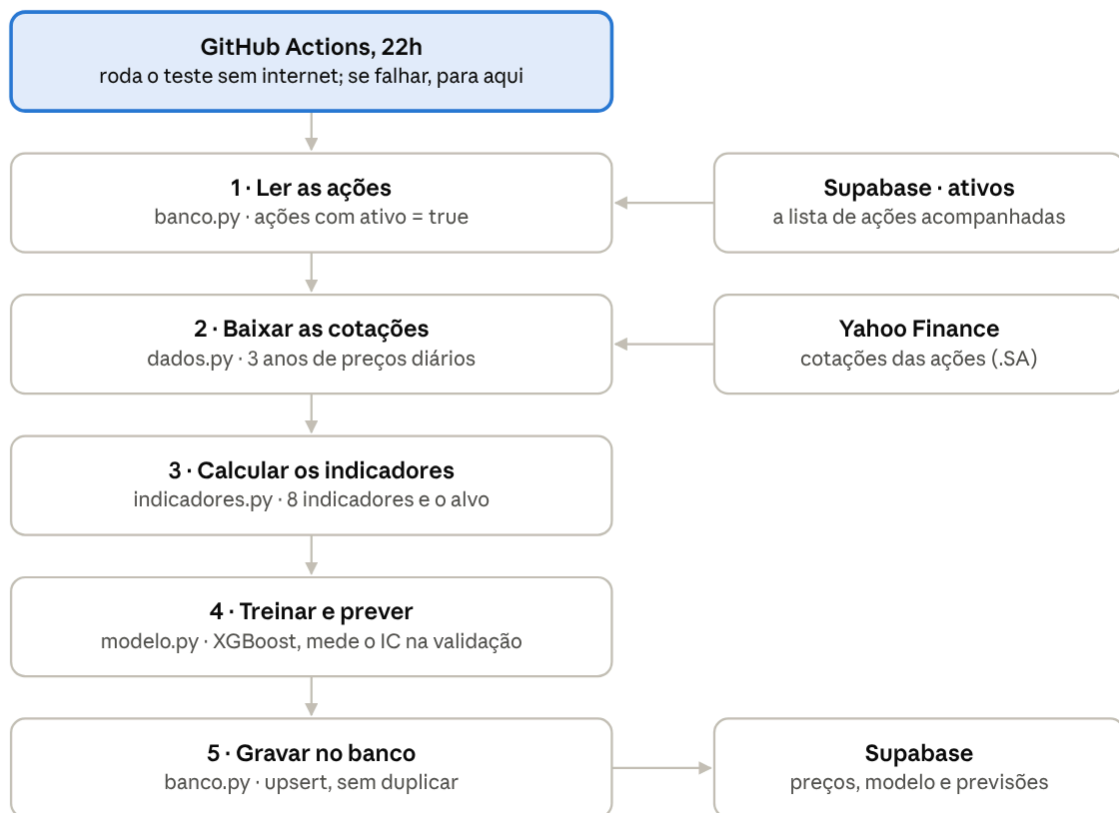
O que o banco oferece	Quem escreve	Quem lê ou chama	Para que serve
Tabelas de mercado: <code>setores</code> , <code>ativos</code> , <code>precos_diarios</code> , <code>modelos</code> , <code>previsoes</code>	Pipeline (e o professor, em <code>ativos</code>)	App, só leitura	Os dados compartilhados por todos
Tabelas do usuário: <code>profiles</code> , <code>carteiras</code> , <code>carteira_itens</code>	App, cada um nas suas linhas	App	O que é de cada pessoa
Views <code>v_ranking_atual</code> e <code>v_desempenho_carteiras</code>	Ninguém (são consultas salvas)	App	Entregam os <code>JOIN</code> s prontos, para o app não precisar montá-los
Função <code>montar_carteira</code>	Ninguém (é código no banco)	App, por RPC	A regra de negócio de criar uma carteira
Trigger <code>criar_perfil</code>	Disparado pelo cadastro	Ninguém chama diretamente	Cria o perfil de cada usuário novo
Bucket <code>avatares</code>	App, cada um na sua pasta	App	Guarda as fotos; o banco guarda só o link

Um exercício que mostra o contrato na prática: renomeie a coluna `retorno_previsto`. O pipeline passa a falhar ao gravar e o app ao ler, embora nenhum dos dois tenha mudado uma linha. Por isso o schema muda pouco e com cuidado.

O pipeline Python

O pipeline é uma receita de 5 etapas, uma por arquivo, que começa e termina no banco: lê a lista de ações, e no fim grava preços, modelo e previsões.

Cinco etapas, do banco ao banco: lê a lista de ações e grava as previsões



pipeline · 5 etapas, 1 arquivo cada

O teste roda antes de qualquer etapa: se ele falhar, o banco não recebe nada naquele dia.

Os indicadores técnicos

O modelo não vê preços crus; vê 8 resumos do comportamento recente de cada ação, todos calculados só com preços até o dia em questão.

Indicador	O que mede
retorno_5d , retorno_21d , retorno_63d	Quanto a ação subiu ou caiu na última semana, mês e trimestre
volatilidade_21d	O quanto o preço oscilou no último mês
rsi_14	Força relativa, de 0 a 100: perto de 100 após muitas altas seguidas
distancia_mm20 , distancia_mm50	Quanto o preço está acima ou abaixo da média dos últimos 20 e 50 pregões
volume_relativo	Se o volume negociado hoje está acima ou abaixo do normal do mês

O **alvo**, aquilo que o modelo aprende a prever, é o retorno dos 5 pregões seguintes. É a única coluna que olha para o futuro.

Por que a validação separa o tempo

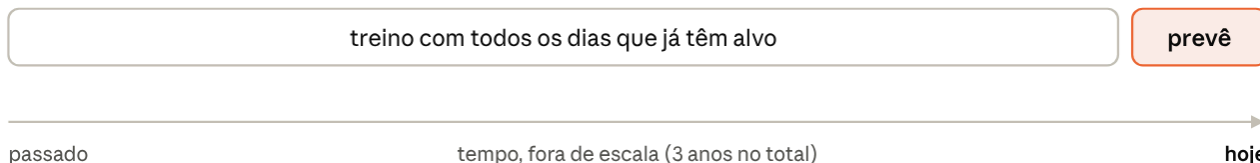
Para medir se o modelo funciona, ele é treinado com o passado e avaliado nos 30 pregões mais recentes, que ele nunca viu. Entre os dois períodos fica um intervalo de 5 pregões: sem ele, o alvo dos últimos dias de treino (que olha 5 dias à frente) já conteria preços do período de avaliação, e o modelo pareceria melhor do que é. Embaralhar os dados, como se faz em muitos problemas de ML, cometeria o mesmo erro em escala maior.

O modelo é avaliado em dias que nunca viu, com 5 pregões de intervalo

1. Medir a qualidade (IC)



2. Modelo final e previsão



divisão do tempo · validação e modelo final

Os últimos 5 pregões ainda não têm alvo, porque o futuro deles não aconteceu: é justamente para o último deles, hoje, que o modelo final faz a previsão.

Gravar sem duplicar

Todas as gravações são *upsert*: insere a linha se ela não existe, atualiza se existe, usando a chave primária (`ticker` + `data` nos preços). Rodar o pipeline duas vezes no mesmo dia não duplica nada, e uma execução que falhou pode simplesmente ser repetida.

O frontend Streamlit

O app Streamlit é um script Python que roda **de novo, do começo ao fim, a cada clique**. Não há eventos nem callbacks para entender: o usuário mexe num filtro, o script reexecuta, refaz as consultas e redesenha a tela com os valores novos.

Esse modelo deixa cada página curta e fácil de ler: consultar o banco, transformar em tabela, mostrar.

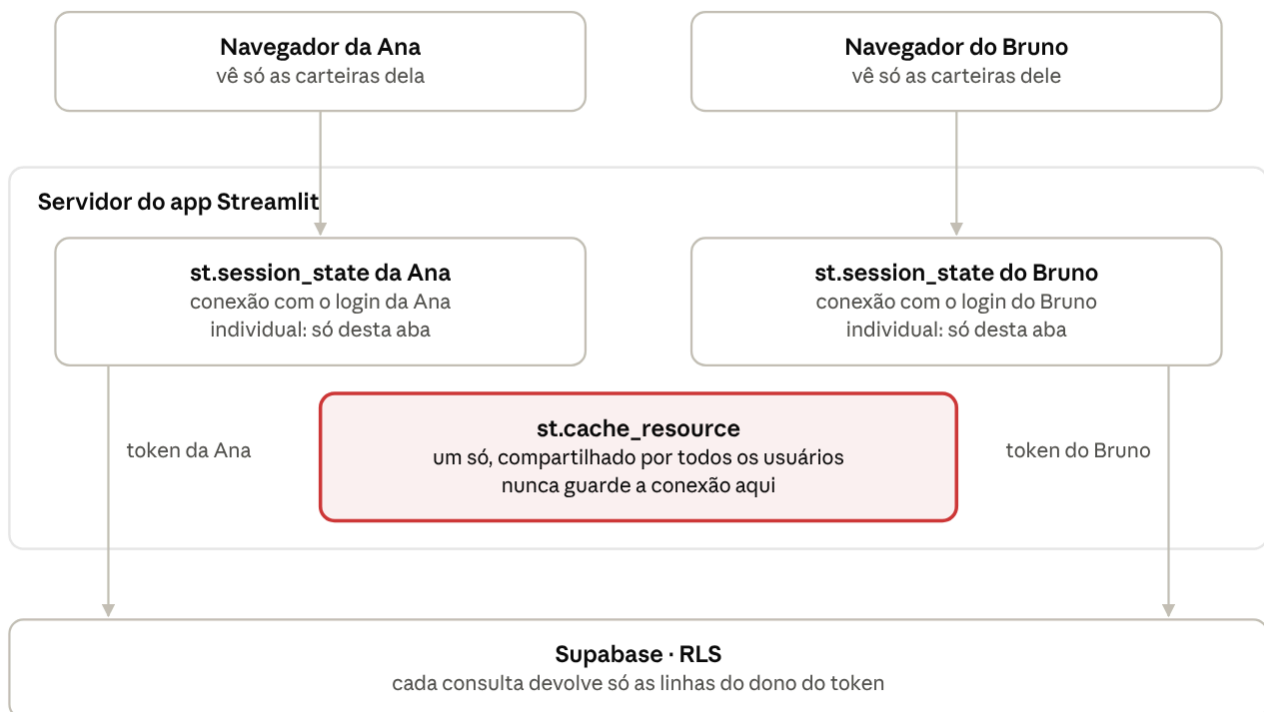
Página	Lê de	Escreve em
Ranking	View <code>v_ranking_atual</code>	—
Ação	<code>ativos</code> (com o setor por <code>JOIN</code>), <code>precos_diarios</code> , <code>previsoes</code>	—
Montar carteira	—	Função <code>montar_carteira</code> , por RPC
Minhas carteiras	View <code>v_desempenho_carteiras</code> , <code>carteira_itens</code>	<code>carteiras</code> (excluir)
Modelo	<code>modelos</code>	—
Meu perfil	<code>profiles</code>	<code>profiles</code> e o bucket <code>avatares</code>

O que sobrevive à reexecução

Se o script recomeça a cada clique, algo precisa guardar quem está logado. O Streamlit oferece dois lugares, e escolher o errado é a falha de segurança mais séria que este app poderia ter:

- `st.session_state` é **individual**: cada aba de navegador tem o seu. É onde fica a conexão com o Supabase, com o login de cada usuário.
- `st.cache_resource` é **compartilhado por todos os usuários** do app. Ótimo para coisas iguais para todos, como um modelo carregado. Se a conexão ficasse aqui, o segundo visitante herdaria o login do primeiro e veria as carteiras dele.

Cada usuário tem a sua conexão; o cache compartilhado nunca pode guardá-la



sessões no Streamlit · 2 usuários, 1 cache compartilhado

Cada token só sai da sessão do seu dono; a caixa em vermelho é o lugar onde ele nunca pode ficar.

Segurança: duas chaves e a RLS

A segurança não está no código do app: está no banco. O app pode ter um erro, ou alguém pode chamar o Supabase direto com a mesma chave pública, e mesmo assim só verá o que as políticas de RLS permitem.

Chave	Quem usa	Onde fica	O que pode
Publishable (<code>sb_publishable_...</code>)	App Streamlit	Secrets do Streamlit; pode até ser pública	Só o que as políticas de RLS liberam para o usuário logado
Secreta (<code>sb_secret_...</code>)	Pipeline Python	Secrets do GitHub, e nenhum outro lugar	Tudo: ignora a RLS

No login, o Supabase devolve ao app um **token** que identifica o usuário. A partir daí, toda consulta leva esse token, e o PostgreSQL acrescenta por conta própria a condição da política, como `user_id = auth.uid()`, a cada consulta. O app pede "todas as carteiras" e recebe só as dele.

A RLS filtra cada consulta do app; só a chave secreta passa por fora

Usuário logado, pelo app



Pipeline, com a chave secreta



caminho de uma consulta · com e sem RLS

As duas linhas passam pelos mesmos lugares; o que muda o resultado é só a chave, conferida pela API.

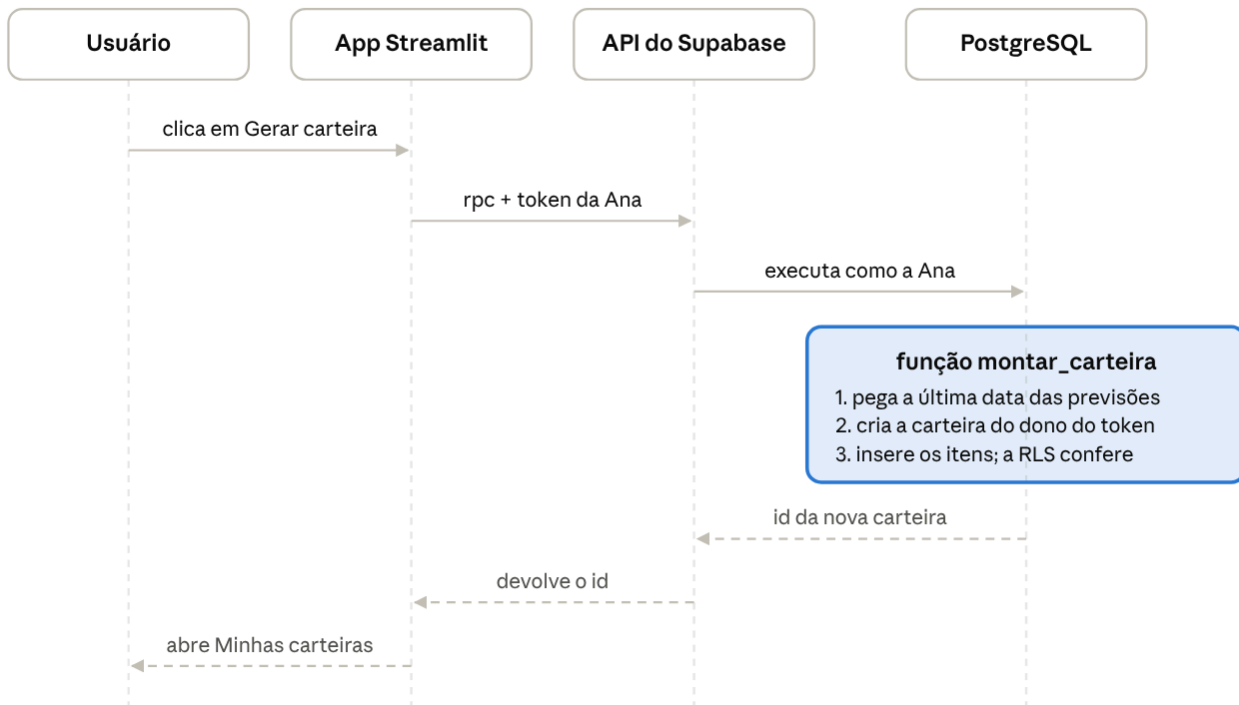
Quem	Tabelas de mercado	Suas carteiras	Carteiras dos outros	Fotos
Visitante sem login	Nada	—	Nada	Vê pelo link, não envia
Usuário logado, pelo app	Lê	Lê, cria e apaga	Nada	Envia só na própria pasta
Pipeline, com a chave secreta	Lê e grava	Tudo	Tudo	Tudo

O pipeline pode tudo, e é exatamente por isso que a chave dele nunca sai dos secrets do GitHub.

Uma ação de ponta a ponta: "Gerar carteira"

Um clique no botão "Gerar carteira" atravessa todas as camadas, e o app escreve uma única linha de código para isso: `cliente().rpc("montar_carteira", {...})`.

A regra de negócio roda dentro do banco; o app só faz uma chamada



sequência do clique · 4 participantes, 6 mensagens

Setas cheias são pedidos e tracejadas, respostas; tudo o que decide algo acontece na caixa azul, dentro do PostgreSQL.

Três coisas valem observar nesse caminho. A regra de negócio (pegar as N melhores do último pregão, com pesos iguais) mora numa função do banco, não no app. A função roda com as permissões de quem clicou, então a RLS também vale lá dentro. E o `user_id` da carteira nunca viaja do app: o banco o preenche com `auth.uid()`, o dono do token.

As ideias para levar

O projeto é pequeno, mas as decisões de arquitetura são as mesmas de sistemas grandes.

1. **O banco é o contrato.** Pipeline e app só combinam nomes de tabelas, colunas e funções; cada um pode ser reescrito sem tocar no outro.
2. **Regras e segurança ficam no banco.** RLS, chaves estrangeiras, `check` e a função `montar_carteira` valem para qualquer cliente, inclusive um que ainda não existe.
3. **Cada programa com a chave mínima de que precisa.** O app só tem a publishable; a secreta vive num único lugar.
4. **Operações repetíveis.** Upsert e chaves primárias tornam seguro rodar o pipeline de novo depois de uma falha.

5. **Configuração em dados, não em código.** A lista de ações está na tabela `ativos` :
mudar o que o sistema acompanha não exige deploy.
6. **Testar antes de gravar.** O workflow roda o teste sem internet antes do pipeline; se ele falhar, nada chega ao banco.
7. **Avaliar respeitando o tempo.** Em dados que evoluem no tempo, treina-se com o passado e avalia-se no futuro, com um intervalo entre os dois.

Para explorar em aula

Cada exercício mexe em uma peça e mostra o efeito nas outras.

- ☐ **Contrato:** em `v_ranking_atual`, acrescente a coluna `volume` (do último pregão). O que precisa mudar no app para exibi-la? E no pipeline?
- ☐ **Configuração em dados:** insira uma ação nova em `ativos` pelo Table Editor e rode o workflow manualmente. Em que tela ela aparece primeiro?
- ☐ **RLS:** com dois usuários, A e B, monte uma carteira com cada um. No SQL Editor, por que `select * from carteiras` mostra as duas, e no app cada um vê só a sua?
- ☐ **Cache:** no `conexao.py`, troque `st.session_state` por `@st.cache_resource` e entre com dois navegadores. Descreva o que acontece e por que é grave.
- ☐ **Regra de negócio:** altere `montar_carteira` para aceitar no máximo 2 ações por setor. Precisou mudar alguma linha do app?
- ☐ **Idempotência:** rode o workflow duas vezes seguidas. Conte as linhas de `previsoes` antes e depois e explique o resultado.
- ☐ **Validação temporal:** em `modelo.py`, retire o intervalo de 5 pregões entre treino e validação. O IC sobe ou desce? Por que um número melhor aqui é uma má notícia?